

Algorithm Complexity & Mathematical Functions: Complete Guide

Table of Contents

- 1. [Introduction to Complexity Analysis](#)
- 2. [Growth Rates & Big-O Notation](#)
- 3. [Mathematical Functions](#)
- 4. [Complexity Classes](#)
- 5. [Common Algorithms & Their Complexity](#)
- 6. [Data Structure Complexities](#)
- 7. [ML/AI Algorithm Complexity](#)
- 8. [Space Complexity](#)
- 9. [Amortized Analysis](#)
- 10. [Interview Questions \(50\)](#)

1. Introduction to Complexity Analysis

What is Time Complexity?

Definition: Measure of how the runtime of an algorithm grows as the input size increases.

Example: Finding max element in array

Input size: $n = 5$
Operations: 5 comparisons

Input size: $n = 1000$
Operations: 1000 comparisons

Pattern: Operations grow linearly with input size
→ Time Complexity: $O(n)$

Why Does It Matter?

Sorting 1 million items:

Algorithm	Time Complexity	Actual Time
Bubble Sort	$O(n^2)$	~11.5 days
Merge Sort	$O(n \log n)$	~20 seconds
Built-in Sort	$O(n \log n)$	~1 second

10x difference in implementation matters!

Big-O Notation

Formal Definition:

$f(n) = O(g(n))$ if there exist constants c and n_0 such that:
 $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

In plain English:
 $f(n)$ grows no faster than $g(n)$ (ignoring constant factors)

Example:

$f(n) = 3n^2 + 5n + 2$

Breakdown:

- $3n^2$: Dominates for large n
- $5n$: Becomes insignificant
- 2 : Constant, irrelevant

Result: $f(n) = O(n^2)$

Why ignore constants?

$n = 1,000,000$:

- $3n^2 = 3,000,000,000,000$
- $5n = 5,000,000$
- $2 = 2$

$3n^2$ dominates completely!

Other Asymptotic Notations

Notation	Meaning
$O(g(n))$	Upper bound (worst case) $f(n) \leq c \cdot g(n)$
$\Omega(g(n))$	Lower bound (best case) $f(n) \geq c \cdot g(n)$
$\Theta(g(n))$	Tight bound (average case) $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$
$o(g(n))$	Strict upper bound $f(n) < c \cdot g(n)$
$\omega(g(n))$	Strict lower bound

$f(n) > c \cdot g(n)$

Example:

```
# Binary Search
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

# Best case: Ω(1) - Found at first try
# Worst case: O(log n) - Not found
# Average case: Θ(log n)
```

2. Growth Rates & Big-O Notation

Complete Hierarchy

Complexity	Name	Example
$O(1)$	Constant	Array access
$O(\log \log n)$	Double Log	Interpolation search (uniform)
$O(\log n)$	Logarithmic	Binary search
$O(\sqrt{n})$	Square Root	Primality testing (trial division)
$O(n)$	Linear	Array traversal
$O(n \log n)$	Linearithmic	Merge sort, Heap sort
$O(n \log^2 n)$	Linearithmic ²	Some divide & conquer
$O(n^2)$	Quadratic	Bubble sort, Selection sort
$O(n^3)$	Cubic	Matrix multiplication (naive)
$O(n^4)$	Quartic	4 nested loops
$O(2^n)$	Exponential	Recursive Fibonacci
$O(n!)$	Factorial	Generate all permutations
$O(n^n)$	N to N	Extreme brute force
$O(2^{(2^n)})$	Double Exp	Tower of Hanoi variations

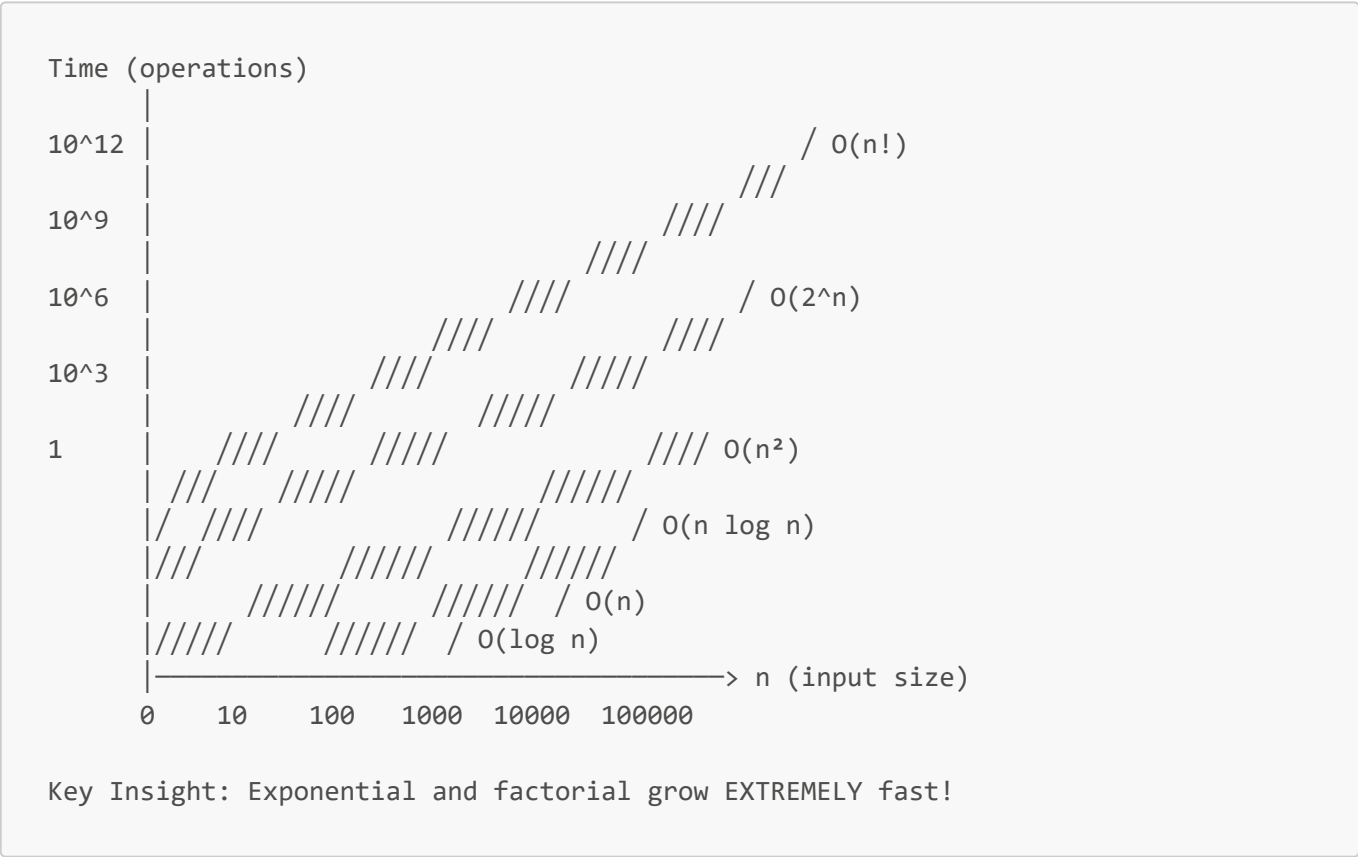
Visual Comparison (Growth Rates)

Operations for different n values:

n	O(1)	O(log n)	O(n)	O(n log n)	O(n²)	O(2^n)	O(n!)
1	1	0	1	0	1	2	1
10	1	3	10	33	100	1,024	3.6M
100	1	7	100	664	10K	1.3×10³⁰	9.3×10¹⁵⁷
1000	1	10	1K	9,966	1M	∞	∞
10K	1	13	10K	130K	100M	∞	∞
100K	1	17	100K	1.7M	10B	∞	∞
1M	1	20	1M	20M	1T	∞	∞

Legend:
K = thousand
M = million
B = billion
T = trillion
∞ = Practically infinite (takes longer than universe age)

Growth Rate Graphs (ASCII Art)



Detailed Growth Analysis

O(1) - Constant Time

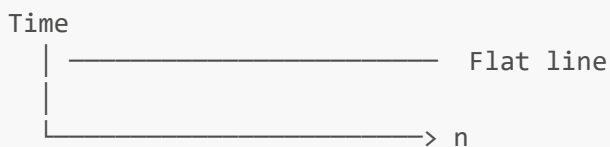
Definition: Operations take same time regardless of input size.

```
def get_first_element(arr):
    return arr[0] # Always 1 operation

# Examples
arr1 = [1, 2, 3] # 1 operation
arr2 = [1, 2, ..., 1M] # Still 1 operation!

# Common O(1) operations:
- Array indexing: arr[5]
- Hash table lookup: dict[key]
- Stack push/pop
- Math operations: a + b, a * b
- Variable assignment: x = 5
```

Graph:



$O(\log n)$ - Logarithmic Time

Definition: Cuts problem size in half each step.

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

# Why O(log n)?
# Each iteration halves the search space:
# n → n/2 → n/4 → n/8 → ... → 1
# Number of steps = log2(n)

# Examples:
n = 1024 → log2(1024) = 10 steps
n = 1,000,000 → log2(1,000,000) ≈ 20 steps
n = 1,000,000,000 → log2(1,000,000,000) ≈ 30 steps
```

```
# Common  $O(\log n)$  operations:
- Binary search (sorted array)
- Balanced BST operations (insert, search, delete)
- Finding GCD (Euclidean algorithm)
```

Intuition:

Finding a name in phone book (1000 pages):

- Linear search: Check page 1, 2, 3... → 500 pages average
- Binary search: Open middle, go left/right → ~10 pages!

Steps:

1000 pages
 ↓ (open middle)
 500 pages
 ↓ (go left/right)
 250 pages
 ↓
 125 → 63 → 32 → 16 → 8 → 4 → 2 → 1
 Total: 10 steps!

 $O(n)$ - Linear Time

Definition: Operations proportional to input size.

```
def find_max(arr):
    max_val = arr[0]
    for num in arr: # Visit each element once
        if num > max_val:
            max_val = num
    return max_val

# Number of operations = n
# Where n = length of array

# Common  $O(n)$  operations:
- Array traversal
- Finding min/max
- Linear search
- Counting elements
- Summing array
```

Examples:

```
#  $O(n)$  - Single loop
def sum_array(arr):
```

```

total = 0
for num in arr: # n iterations
    total += num
return total

# Still O(n) - Multiple passes (2n = O(n))
def find_min_max(arr):
    min_val = arr[0]
    max_val = arr[0]

    for num in arr: # First pass
        if num < min_val:
            min_val = num

    for num in arr: # Second pass
        if num > max_val:
            max_val = num

    return min_val, max_val
# Total: 2n operations, still O(n)

```

O(n log n) - Linearithmic Time

Definition: Combination of linear and logarithmic.

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid]) # Divide (log n levels)
    right = merge_sort(arr[mid:]) # Divide (log n levels)

    return merge(left, right) # Merge (n operations per level)

# Why O(n log n)?
# - log n levels of recursion (dividing)
# - n operations at each level (merging)
# - Total: n × log n

# Visualization:
#           [8,3,5,4,2,6,1,7] ← n elements
#           /               \
#       [8,3,5,4]         [2,6,1,7] ← n elements total
#       /   \             /   \
#   [8,3] [5,4]       [2,6] [1,7] ← n elements total
#   / \  / \         / \   / \
#  8 3 5 4       2 6 1 7 ← n elements total
#
# Height = log n
# Work per level = n

```

```
# Total = n log n

# Common O(n log n) algorithms:
- Merge sort
- Heap sort
- Quick sort (average case)
- Sorting algorithms (optimal comparison-based)
```

$O(n^2)$ - Quadratic Time

Definition: Nested loops, each iterating n times.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):          # Outer loop: n iterations
        for j in range(n - i - 1): # Inner loop: ~n iterations
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# Operations:  $n \times n = n^2$ 

# Common  $O(n^2)$  operations:
- Bubble sort, Selection sort, Insertion sort
- Nested loops over same data
- Comparing all pairs
- Naive matrix multiplication
```

Examples:

```
#  $O(n^2)$  - All pairs
def find_all_pairs_sum(arr, target):
    pairs = []
    for i in range(len(arr)):          # n iterations
        for j in range(i + 1, len(arr)): # n iterations
            if arr[i] + arr[j] == target:
                pairs.append((arr[i], arr[j]))
    return pairs
#  $n \times n = n^2$ 

#  $O(n^2)$  - Matrix operations
def matrix_add(A, B):
    n = len(A)
    result = [[0] * n for _ in range(n)]
    for i in range(n):          # n rows
        for j in range(n):      # n columns
            result[i][j] = A[i][j] + B[i][j]
    return result
#  $n \times n = n^2$ 
```


$O(n^3)$ - Cubic Time

Definition: Three nested loops.

```
def matrix_multiply(A, B):
    n = len(A)
    result = [[0] * n for _ in range(n)]

    for i in range(n):          # n iterations
        for j in range(n):      # n iterations
            for k in range(n):  # n iterations
                result[i][j] += A[i][k] * B[k][j]

    return result
# n × n × n = n³

# Common  $O(n^3)$  operations:
- Naive matrix multiplication
- Floyd-Warshall algorithm (all-pairs shortest path)
- Three nested loops
```

$O(2^n)$ - Exponential Time

Definition: Doubles with each additional element.

```
def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

Why $O(2^n)$?
Tree of recursive calls:
fib(5)
/ \
fib(4) fib(3)
/ \
fib(3) fib(2) fib(2) fib(1)
/ \
fib(2) fib(1) ...

Each level roughly doubles the number of calls
Height = n
Nodes $\approx 2^n$

Growth:
fib(10) = 177 calls
fib(20) = 21,891 calls
fib(30) = 2,692,537 calls

```
fib(40) = 331,160,281 calls (takes minutes!)
```

```
# Common  $O(2^n)$  problems:
```

- Recursive Fibonacci
- Tower of Hanoi
- Generating all subsets
- Brute force combinatorial problems

Why Exponential is BAD:

```
n = 20: ~1 million operations (1 second)
n = 30: ~1 billion operations (16 minutes)
n = 40: ~1 trillion operations (12 days)
n = 50: ~1 quadrillion operations (35 years!)
```

$O(n!)$ - Factorial Time

Definition: $n \times (n-1) \times (n-2) \times \dots \times 1$ operations.

```
def generate_permutations(arr):
    if len(arr) <= 1:
        return [arr]

    perms = []
    for i in range(len(arr)):
        rest = arr[:i] + arr[i+1:]
        for perm in generate_permutations(rest):
            perms.append([arr[i]] + perm)

    return perms

# Number of permutations of n elements = n!
# Example:
# n = 3: [1,2,3] → 6 permutations (3! = 6)
# n = 4: [1,2,3,4] → 24 permutations (4! = 24)
# n = 10: → 3,628,800 permutations
# n = 13: → 6,227,020,800 permutations (6 billion!)

# Common  $O(n!)$  problems:
# - Traveling Salesman Problem (brute force)
# - Generating all permutations
# - Solving lock combinations (n positions)
```

Factorial Growth:

```
1! = 1
5! = 120
```

$10! = 3,628,800$
 $15! = 1,307,674,368,000$ (1.3 trillion)
 $20! = 2.4 \times 10^{18}$ (would take millions of years to compute)

Comparison Table

Complexity	n=10	n=100	n=1000	n=1000000
$O(1)$	1	1	1	1
$O(\log n)$	3	7	10	20
$O(n)$	10	100	1K	1M
$O(n \log n)$	33	664	10K	20M
$O(n^2)$	100	10K	1M	1T
$O(n^3)$	1K	1M	1B	10^{18}
$O(2^n)$	1K	10^{30}	∞	∞
$O(n!)$	3.6M	10^{157}	∞	∞

✓ = Acceptable (< 1 second)

⚠ = Slow (1-60 seconds)

X = Impractical (> 1 minute)

∞ = Impossible (longer than universe age)

Rules for Calculating Complexity

Rule 1: Drop Constants

$$3n + 5 = O(n)$$

$$100n = O(n)$$

$$n/2 = O(n)$$

Why? For large n, constants don't matter:

$$n = 1,000,000:$$

$$3n = 3,000,000$$

$$100n = 100,000,000$$

Both are still linear!

Rule 2: Drop Lower Order Terms

$$n^2 + n = O(n^2)$$

$$n^3 + n^2 + n = O(n^3)$$

$$2^n + n^3 = O(2^n)$$

Why? Higher order dominates:

$$n = 1000:$$

$$n^2 = 1,000,000$$

```
n = 1,000
n2 dominates!
```

Rule 3: Different Variables Stay

```
f(n, m) = n + m = O(n + m)
f(n, m) = n × m = O(n × m)
```

Can't simplify further without knowing relationship between n and m

Rule 4: Sequential Operations Add

```
def func(arr):
    for x in arr: # O(n)
        print(x)

    for x in arr: # O(n)
        print(x * 2)

# Total: O(n) + O(n) = O(2n) = O(n)
```

Rule 5: Nested Operations Multiply

```
def func(arr):
    for i in arr: # O(n)
        for j in arr: # O(n)
            print(i, j)

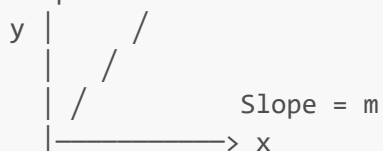
# Total: O(n) × O(n) = O(n2)
```

3. Mathematical Functions

Linear Functions

Form: $f(x) = mx + b$

Graph:



Properties:

- Constant rate of change

- Straight line
- Slope = m (rise/run)
- Y-intercept = b

Examples:

```
# Linear growth
def linear(x):
    return 2*x + 3

# Values:
x: 0  1  2  3  4  5
y: 3  5  7  9  11 13

Growth rate: +2 per step (constant)

# In algorithms:
# Single loop = O(n) = linear
for i in range(n): # f(n) = n
    process(i)
```

Applications:

- Linear search time
- Array traversal
- Simple iterations
- Cost functions in ML

Polynomial Functions

General Form: $f(x) = a_n x^n + a_{n-1} x^{(n-1)} + \dots + a_1 x + a_0$

Quadratic (n=2)

Form: $f(x) = ax^2 + bx + c$

Graph:



Properties:

- Parabola shape
- Vertex at $x = -b/(2a)$
- Grows faster than linear

Examples:

```
# Quadratic growth
def quadratic(x):
    return x**2

# Values:
x:  0  1  2  3  4  5
y:  0  1  4  9 16 25

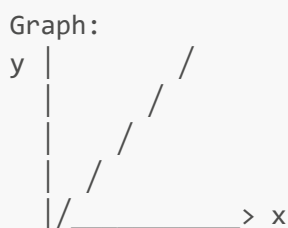
Growth rate: Increasing (1, 3, 5, 7, 9...)

# In algorithms:
# Nested loops = O(n²) = quadratic
for i in range(n):
    for j in range(n):
        process(i, j)

# Practical example:
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):           # n iterations
        for j in range(n-i-1):  # ~n iterations
            if arr[j] > arr[j+1]:
                swap(arr[j], arr[j+1])
# Time: O(n²)
```

Cubic (n=3)

Form: $f(x) = ax^3 + bx^2 + cx + d$



Growth rate: Much faster than quadratic

Examples:

```
# Cubic growth
def cubic(x):
    return x**3

# Values:
x:  0  1  2  3  4  5
y:  0  1  8 27 64 125
```

```
# In algorithms: O(n³)
for i in range(n):
    for j in range(n):
        for k in range(n):
            process(i, j, k)

# Practical example: Matrix multiplication
def matrix_multiply(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    # Time: O(n³)
```

Exponential Functions

Form: $f(x) = a^x$ (where $a > 1$)

Graph:



Properties:

- Extremely fast growth
- Doubles (or more) with each step
- Unsustainable for large x

Common Bases:

```
# Base 2: f(x) = 2^x
def exponential_2(x):
    return 2**x

# Values:
x: 0  1  2  3  4  5  10  20
y: 1  2  4  8  16  32  1024  1,048,576

# Base e: f(x) = e^x (natural exponential)
import math
def exponential_e(x):
    return math.exp(x)

# Values:
x: 0  1  2  3  4  5
```

```

y:  1    2.72    7.39    20.09    54.60    148.41

# Base 10: f(x) = 10^x
def exponential_10(x):
    return 10**x

# Values:
x:  0    1    2    3    4    5
y:  1    10   100  1000  10000  100000

```

In Algorithms:

```

# O(2^n) - Recursive Fibonacci
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# Call tree grows exponentially:
#           fib(4)
#         /      \
#       fib(3)   fib(2)
#      /  \    /  \
#    fib(2) fib(1) fib(1) fib(0)
#   /  \
# fib(1) fib(0)
#
# Nodes = 2^n (approximately)

# Practical impact:
fib(10) = 177 calls      (instant)
fib(20) = 21,891 calls   (instant)
fib(30) = 2,692,537 calls (0.3 seconds)
fib(40) = 331,160,281 calls (30 seconds)
fib(50) = ~ 40 billion calls (days!)

```

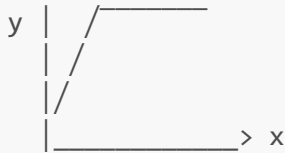
Applications:

- Exponential growth/decay
- Compound interest
- Population growth
- Recursive algorithms (worst case)
- Backtracking algorithms

Logarithmic Functions

Form: $f(x) = \log_b(x)$ (where b is the base)

Graph:



Properties:

- Inverse of exponential
- Slow growth
- Very efficient for large inputs

Common Logarithms:

```
import math

# Base 2: log2(x) (binary logarithm)
def log_2(x):
    return math.log2(x)

# Values:
x:  1  2  4  8  16  32  64  128  1024
y:  0  1  2  3  4  5  6  7  10

# Base e: ln(x) (natural logarithm)
def ln(x):
    return math.log(x)

# Values:
x:  1  2.72  7.39  20.09  54.60
y:  0  1  2  3  4

# Base 10: log10(x) (common logarithm)
def log_10(x):
    return math.log10(x)

# Values:
x:  1  10  100  1000  10000
y:  0  1  2  3  4
```

Logarithm Properties:

```
# Product rule
log(a × b) = log(a) + log(b)

# Quotient rule
log(a / b) = log(a) - log(b)

# Power rule
log(a^b) = b × log(a)
```

```
# Change of base
log_b(x) = log_a(x) / log_a(b)

# Why log2 in CS?
# Because we divide by 2 (binary search, binary trees)
log2(1024) = 10 (10 halvings to reach 1)
```

In Algorithms:

```
# O(log n) - Binary search
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

# Halvings:
n = 1024 → 512 → 256 → 128 → 64 → 32 → 16 → 8 → 4 → 2 → 1
Steps = log2(1024) = 10

# Comparison:
Array size    Linear O(n)    Binary O(log n)
1,000         1,000         10
1,000,000     1,000,000     20
1,000,000,000 1,000,000,000 30

Binary search is MUCH faster!
```

Applications:

- Binary search
- Balanced tree operations
- Divide and conquer algorithms
- Finding depth of tree

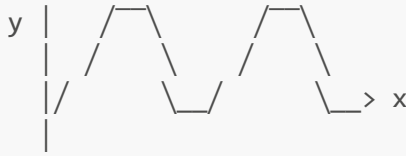
Trigonometric Functions

Sine and Cosine

Form:

- $\sin(x)$: y-coordinate on unit circle
- $\cos(x)$: x-coordinate on unit circle

Graph ($\sin$):



Properties:

- Periodic (repeats every 2π)
- Range: $[-1, 1]$
- $\sin(0) = 0$, $\sin(\pi/2) = 1$, $\sin(\pi) = 0$
- $\cos(0) = 1$, $\cos(\pi/2) = 0$, $\cos(\pi) = -1$

Examples:

```
import math

# Sine function
def sine_values():
    angles = [0, math.pi/6, math.pi/4, math.pi/3, math.pi/2, math.pi]
    for angle in angles:
        print(f"sin({angle:.2f}) = {math.sin(angle):.3f}")

# Output:
sin(0.00) = 0.000
sin(0.52) = 0.500 (30°)
sin(0.79) = 0.707 (45°)
sin(1.05) = 0.866 (60°)
sin(1.57) = 1.000 (90°)
sin(3.14) = 0.000 (180°)

# Cosine function
def cosine_values():
    angles = [0, math.pi/6, math.pi/4, math.pi/3, math.pi/2, math.pi]
    for angle in angles:
        print(f"cos({angle:.2f}) = {math.cos(angle):.3f}")
```

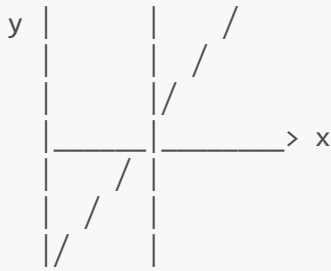
Applications:

- Fourier transforms
- Signal processing
- Physics simulations
- Rotation matrices
- Periodic phenomena

Tangent

Form: $\tan(x) = \sin(x) / \cos(x)$

Graph:



Properties:

- Periodic (repeats every π)
- Undefined at $x = \pi/2 + n\pi$
- Range: $(-\infty, \infty)$

Hyperbolic Functions

Hyperbolic Sine and Cosine

Form:

- $\sinh(x) = (e^x - e^{-x}) / 2$
- $\cosh(x) = (e^x + e^{-x}) / 2$

```
import math
```

```
def hyperbolic_functions(x):
    sinh = math.sinh(x)
    cosh = math.cosh(x)
    tanh = math.tanh(x)

    return sinh, cosh, tanh
```

Values:

x:	-2	-1	0	1	2
sinh:	-3.63	-1.18	0	1.18	3.63
cosh:	3.76	1.54	1	1.54	3.76
tanh:	-0.96	-0.76	0	0.76	0.96

Applications:

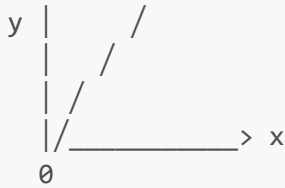
- Machine learning (tanh activation)
- Physics (special relativity)
- Engineering calculations

Special ML/AI Functions

ReLU (Rectified Linear Unit)

Form: $f(x) = \max(0, x)$

Graph:



Properties:

- Linear for $x > 0$
- Zero for $x \leq 0$
- Non-saturating
- Computationally efficient

Implementation:

```
def relu(x):
    return max(0, x)

# Vectorized
import numpy as np
def relu_vectorized(x):
    return np.maximum(0, x)

# Values:
x:  -3  -2  -1   0   1   2   3
y:   0   0   0   0   1   2   3

# Derivative (for backpropagation):
def relu_derivative(x):
    return 1 if x > 0 else 0
```

Variants:

```
# Leaky ReLU
def leaky_relu(x, alpha=0.01):
    return x if x > 0 else alpha * x

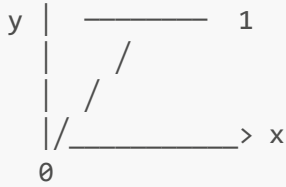
# Parametric ReLU (PReLU)
def prelu(x, alpha):
    return x if x > 0 else alpha * x # alpha is learned

# ELU (Exponential Linear Unit)
def elu(x, alpha=1.0):
    return x if x > 0 else alpha * (np.exp(x) - 1)
```

Sigmoid

Form: $f(x) = 1 / (1 + e^{(-x)})$

Graph:



Properties:

- S-shaped curve
- Range: (0, 1)
- Smooth gradient
- Used for binary classification
- Saturates for large $|x|$

Implementation:

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Values:
x:  -5    -2    -1     0     1     2     5
y:  0.007 0.12  0.27  0.50  0.73  0.88  0.993

# Properties:
sigmoid(0) = 0.5
sigmoid(-x) = 1 - sigmoid(x)
lim(x→∞) sigmoid(x) = 1
lim(x→-∞) sigmoid(x) = 0

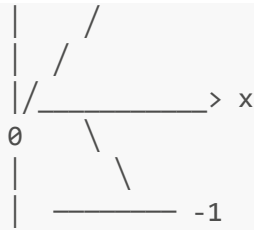
# Derivative (for backpropagation):
def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)
```

Tanh (Hyperbolic Tangent)

Form: $f(x) = (e^x - e^{(-x)}) / (e^x + e^{(-x)})$

Graph:





Properties:

- S-shaped (similar to sigmoid)
- Range: (-1, 1)
- Zero-centered (advantage over sigmoid)
- Still saturates for large $|x|$

Implementation:

```
import numpy as np

def tanh(x):
    return np.tanh(x)
    # Or: (np.exp(x) - np.exp(-x)) / (np.exp(x) + np.exp(-x))

# Values:
x:  -5      -2      -1      0      1      2      5
y: -0.999 -0.964 -0.762 0.000  0.762  0.964  0.999

# Relationship to sigmoid:
# tanh(x) = 2 * sigmoid(2x) - 1

# Derivative:
def tanh_derivative(x):
    t = np.tanh(x)
    return 1 - t**2
```

Softmax

Form: $f(x_i) = e^{(x_i)} / \sum(e^{(x_j)})$

Converts logits to probabilities:

Input (logits): [2.0, 1.0, 0.1]

↓

Softmax: [0.659, 0.242, 0.099]

↓

Sum = 1.0 (probability distribution)

Implementation:

```
import numpy as np

def softmax(x):
    # Subtract max for numerical stability
    exp_x = np.exp(x - np.max(x))
    return exp_x / np.sum(exp_x)

# Example:
logits = np.array([2.0, 1.0, 0.1])
probs = softmax(logits)
print(probs) # [0.659, 0.242, 0.099]
print(np.sum(probs)) # 1.0

# Temperature scaling:
def softmax_temperature(x, temperature=1.0):
    exp_x = np.exp((x - np.max(x)) / temperature)
    return exp_x / np.sum(exp_x)

# High temperature → uniform distribution
# Low temperature → peaky distribution
```

Function Comparison Table

Function	Growth Rate	Range	Use Case
Linear	$O(n)$	$(-\infty, \infty)$	Simple ops
Quadratic	$O(n^2)$	$(-\infty, \infty)$	Nested loops
Cubic	$O(n^3)$	$(-\infty, \infty)$	Matrix ops
Exponential	$O(2^n)$	$(0, \infty)$	Growth models
Logarithmic	$O(\log n)$	$(-\infty, \infty)$	Search
Factorial	$O(n!)$	Discrete	Permutations
Sine/Cosine	Periodic	$[-1, 1]$	Waves, rotation
Tangent	Periodic	$(-\infty, \infty)$	Slopes
ReLU	$O(n)$ for $x > 0$	$[0, \infty)$	Hidden layers
Sigmoid	Smooth	$(0, 1)$	Binary class.
Tanh	Smooth	$(-1, 1)$	Hidden layers
Softmax	Normalized	$(0, 1)$	Multi-class

Derivative Summary (For ML)

```
# Important for backpropagation

# Linear:  $f(x) = mx + b$ 
def linear_derivative(x, m):
```



```
    return m

# Quadratic:  $f(x) = x^2$ 
def quadratic_derivative(x):
    return 2*x

# Exponential:  $f(x) = e^x$ 
def exp_derivative(x):
    return np.exp(x)

# Logarithmic:  $f(x) = \ln(x)$ 
def log_derivative(x):
    return 1/x

# ReLU:  $f(x) = \max(0, x)$ 
def relu_derivative(x):
    return 1 if x > 0 else 0

# Sigmoid:  $f(x) = 1/(1+e^{(-x)})$ 
def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)

# Tanh:  $f(x) = \tanh(x)$ 
def tanh_derivative(x):
    t = np.tanh(x)
    return 1 - t**2
```
